

Assignment 6 – Qt and PySide 6 GUI

Open your IDE, and in the Terminal, run this code to install PySide 6:

pip3 install pyside6

Watch the demo video to see how the user should be able to interact with this GUI application. Also run each Qt PySide6 example file and watch the example code walkthrough.

Submit your work as a PY file.

Objective:

Students will create a PySide6 application that accepts an integer input, validates it, and displays a randomly generated number based on that input. The program must include widgets, layouts, signals/slots, and basic input validation.

Follow the steps below to build your PySide6 GUI application. Use example code for hints. When asking AI, be sure to specify PySide6. If you ask for Qt, it will give you C++ code.

1. Set Up Your Project

- Create a Python file named **asn6.py**.
- At the top of your file, import the required modules.

2. Create the Main Application Window - Define a class that inherits from QMainWindow.

3. Add the Widgets. Remember that you are in a class, so use "self".

Your GUI must contain:

- A help/instruction label that tells the user to enter an integer greater than 2. Call it lblHelp and set the background color to a readable color of your choice.
 - An input box - with text centered. Call it input.
 - An output label for displaying results - with text centered.
 - Call it lblOutput.
 1. Set the label's font to 25, and the background color to a readable color of your choice.
 - A button called btnRand that displays the text "Random Numbers"
-
- Add ALL Widgets to a Vertical Layout. See the layout example file.
 - A QVBoxLayout stacks widgets vertically in the order you add them.

4. Write a Custom Method

Implement a custom method called update_label().

This method should:

Read the input text which comes in as a string because the value was typed into a GUI input box. This is standard. That means that now you should:

CONVERT what was typed into input to a proper integer data type.

That mean you will have to create a variable.

Use try...except to validate the input text to ensure it is a proper integer

If input is not an integer → show a warning

If integer is less than 2 → show a warning

You can use a MessageBox to pop up a warning when the input fails.

Generate a random number, using the regular Python method for that.

Start the generator at 1, and as the upper bound, use the variable holding the integer value you captured from the user input.

Output the random number on lblOutput

5. Connect Signal to Slot

You must attach a signal from the button to a custom method that runs when clicked to update what is displayed on the lblOutPut. Your custom method is update_label() from the previous step.

```
self.some_button.pressed.connect(self.some_method)
```

6. Start the Application Loop and run the app.

Program Requirements Checklist

Your code should include:

- A PySide6 window (QMainWindow)
- QLineEdit for user input
- QLabel instructions and output
- A QPushButton
- A QVBoxLayout
- Input validation using try/except
- Warning dialogs using QMessageBox

- Random number generation using `random.randint()`
- Centered and styled output label

Please also refer to the grading rubric on the next page. Please email me with questions or concerns.

Criterion	Excellent (Full Credit)	Proficient (Minor Issues)	Developing (Noticeable Gaps)	Beginning (Limited/Not Working)	Points
GUI Structure & Layout (QMainWindow, central widget, QVBoxLayout, all widgets added in correct order)	Uses QMainWindow, central widget, and QVBoxLayout correctly; all widgets appear and are ordered logically; window title/geometry set as specified.	Minor layout misalignment or widget order issue; functionality intact.	Layout present but missing 1–2 required widgets or central widget/setCentralWidget() used incorrectly.	Layout missing or broken; app window does not present required controls.	15
Widgets & Configuration (QLabel help, QLineEdit input alignment, output QLabel font/size/alignment, button)	All widgets implemented; input & output labels aligned center; output label font visibly enlarged; styles applied as specified (green help label; lightsteelblue output).	One minor misconfiguration (e.g., missing style or misalignment) but widgets are present.	Multiple misconfigurations (e.g., default font, no alignment, missing style); widgets still visible.	Key widgets missing (e.g., no output label or button).	15
Signals & Slots (Button wired to method)	QPushButton correctly connected (e.g., <code>.pressed.connect(self.update_label)</code> or <code>.clicked.connect(...)</code>); handler exists and runs.	Connection present but slightly off best practice (e.g., wrong signal but still functional).	Intermittent or incorrect connection; runs only in some cases.	No working connection; button does nothing.	10
Input Retrieval & Conversion (<code>QLineEdit.text()</code> ; <code>int()</code> conversion)	Retrieves text and converts to int correctly; clear structure; no runtime errors for valid input.	Works with valid input; occasional edge case overlooked but doesn't crash.	Conversion logic fragile; crashes for common mistakes; not clearly structured.	No conversion; uses hardcoded values or crashes immediately.	10
Validation & Error Handling (ValueError)	Robust validation: rejects non-integers and integers < 2; uses	Validation works with minor message wording/flow issues	Partial validation (handles either non-integer or	No validation; crashes or	20

Criterion	Excellent (Full Credit)	Proficient (Minor Issues)	Developing (Noticeable Gaps)	Beginning (Limited/Not Working)	Points
catch; integer > 2 requirement; message dialogs)	QMessageBox.warning with helpful messages; clears/reset fields appropriately and returns early.	(e.g., doesn't clear field).	range, not both); messages unclear or missing.	accepts invalid input silently.	
Random Number Generation (random.randint(1, user_int); result shown in label)	Uses random.randint(1, user_int); result displayed immediately in the output label; no blocking calls.	Logic correct but minor formatting issues (e.g., extra text or missing f-string).	Range off by one or incorrect bounds; still displays a number.	Not implemented or incorrect approach (e.g., constant value).	10
Code Quality & Readability (organization, naming, comments)	Clean, readable code; logical structure; minimal but meaningful comments; no dead code or unused imports.	Generally readable; a few minor style issues or leftover commented lines.	Cluttered or inconsistent; significant commented-out code; confusing naming.	Disorganized; hard to follow; numerous unused code blocks.	10
UI Polish & Specification Adherence (title/geometry, alignment, fonts, styles)	Matches spec closely (title "Assignment 6", geometry reasonable, centered text, styled labels); feels polished.	Mostly matches spec; small deviations that don't affect usability.	Multiple spec deviations; still usable but unpolished.	Largely ignores spec; poor UX.	5
Program Execution (starts, shows window, no crashes on typical use)	Runs without errors; handles typical user actions gracefully; clean exit.	Runs with a minor warning/edge-case glitch.	Runs but with frequent hiccups or crashes under common mis-inputs.	Fails to run or immediately crashes.	5

Total: 100 points

Tips for Testing:

- **Edge cases to test:** empty input, non-numeric text, negative numbers, 0, 1, and a valid input like 7.
- **Early return pattern:** After showing a `QMessageBox.warning`, return from the handler to avoid running the random-number code on invalid input.
- **Alignment flags:** Use bitwise OR (`Qt.AlignHCenter | Qt.AlignVCenter`) to center content in `QLineEdit/QLabel`.